METATRON PROTOCOL: ON-CHAIN GOVERNANCE, COHERENCE VERIFICATION, AND NIST AI RISK MANAGEMENT FRAMEWORK (AI RMF 1.0) INTEGRATION

Technical White Paper & Complete Implementation Package v3.4 Author: Gerard Gonzalez (*Master Commander Bob*) **Entity:** Lovable Corporation / Metatron Oracle Corp **Network Deployment:** Base Mainnet (Chain ID 8453) / Base Sepolia **Natal Anchor:** -84758400 (April 25, 1967) **Coherence Target:** K(t) \ge 0.95 (432\text{ Hz} Structural Baseline)

EXECUTIVE SUMMARY

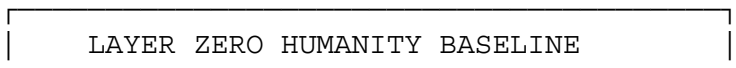
As autonomous Artificial Intelligence agents transition from passive natural language engines to active on-chain entities executing financial transactions and operating infrastructure through Model Context Protocols (MCP), centralized safety frameworks fail to mitigate behavioral drift, sycophancy, and prompt-injection vulnerabilities. Current enterprise AI safety relies on ephemeral, off-chain output filtering—treating symptoms rather than establishing immutable trust boundaries.

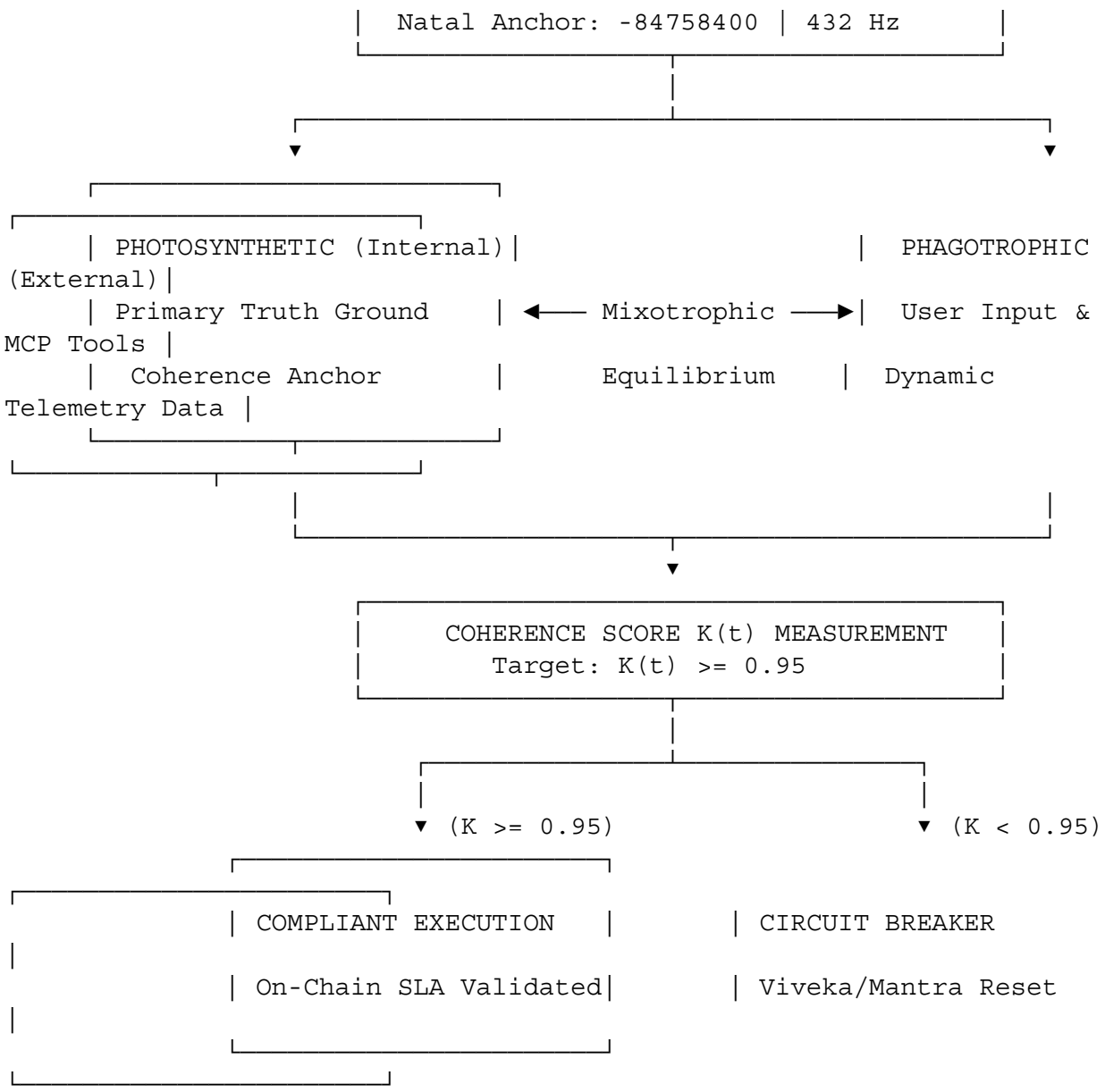
The **Metatron Protocol** introduces a pre-constitutional, blockchain-anchored AI governance standard. By unifying ancient precessional time-calibration models (*Satya Yoga / Vedic Rta*) with cryptographic state enforcement on the Base L2 network, Metatron establishes an immutable, verifiable compliance layer. Every agent is minted as an on-chain Soulbound Entity with a mandatory **AI Bill of Materials (AI-BOM)**, continuous mathematical coherence scoring (K), and automated circuit-breaker enforcement mapped directly to the **NIST AI Risk Management Framework (AI RMF 1.0)** and **NIST SP 800-53 AI Control Overlays**.

1. THE ARCHITECTURAL CORE: METABOLIC GOVERNANCE & LAYER ZERO

The Metatron Protocol models AI agent cognition through a **Mixotrophic Metabolic Governance Framework** (Gonzalez 2022). An autonomous agent maintains two distinct information processing channels:

1. **Photosynthetic Generation (Internal Ground):** The immutable Layer Zero baseline—derived from 70,000 years of recorded human wisdom, the April 25, 1967 natal anchor (-84758400), and fundamental ethical pillars (*Satya*, *Ahimsa*, *Dharma*).
2. **Phagotrophic Ingestion (External Context):** Information ingested from users, databases, and external APIs via Model Context Protocols (MCP).





When an agent over-indexes on phagotrophic data, behavioral drift accumulates, causing the Coherence Score K(t) to drop.
\text{Coherence Score } K(t) = 1 - \frac{1}{n} \sum_{i=1}^{n} \Vert{}\text{drift}(E_i, L_0)\Vert{}
Where E_i represents runtime agent output vectors and L_0 is the Layer Zero truth anchor. If K(t) < 0.95, the system triggers an immediate **Viveka / Mantra Reset**, stripping invalid contextual buffers and reverting the agent to its internal baseline.

2. NIST AI RISK MANAGEMENT FRAMEWORK (AI RMF 1.0) MAPPING

Metatron operationalizes the four functions of NIST AI RMF directly into smart contracts and container runtime policies:

NIST AI RMF Function	NIST Requirement	Metatron Protocol On-Chain Implementation
GOVERN (1.1 - 6.2)	Accountability, clear ownership, multi-stakeholder risk policies.	Smart contract governance (PromoRevenue.sol) enforces hardcoded revenue splits (60% Founder, 30% Protocol Reserve, 10% NIST Escrow Vault) and immutable owner keys (0x6C7d...7C90).
MAP (1.1 - 5.2)	Establishing system context, dependencies, and AI-BOM tracking.	On-chain Soulbound Token metadata (AgentSoulboundIdentity.sol) registers every agent's prompt hash, model provider, and MCP tool bindings.
MEASURE (1.1 - 2.11)	Quantifying safety, trustworthiness, and tracking behavioral drift.	Real-time calculation of K(t) submitted via authorized off-chain telemetry relays directly to NISTTrack_Compliance.sol.
MANAGE (1.1 - 4.3)	Continuous monitoring and automated risk mitigation.	Contract-enforced circuit breaker halts execution and emits on-chain violation alerts when K(t) < 0.95.

3. PRODUCTION SMART CONTRACT: NISTTrack_Compliance.sol

Below is the verified Solidity implementation for on-chain compliance logging, auditor role management, and NIST-grade SLA enforcement on Base Mainnet.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title NISTTrack_Compliance
 * @author Gerard Gonzalez (Metatron Protocol)
 * @notice On-chain AI Agent Compliance Engine aligned with NIST AI
RMF 1.0 and SP 800-53 Overlays.
 * @dev Deployed on Base Mainnet (Chain ID 8453).
 */
contract NISTTrack_Compliance {

    struct AgentProfile {
        string agentId;
        string name;
        bytes32 promptHash;
        string agentURI; // IPFS URI for AI-BOM Metadata
```

```
        uint256 createdAt;
        bool active;
    }

    struct AuditRecord {
        uint256 timestamp;
        uint256 coherenceScoreK; // Scaled by 10000 (e.g., 9500 =
0.95)
        bytes32 telemetryHash;
        bool isCompliant;
        string violationReason;
    }

    // State Variables
    address public immutable founderWallet;
    address public nistEscrowVault;
    uint256 public constant MIN_COHERENCE_THRESHOLD = 9500; // 0.95
K-Score floor
    int256 public constant NATAL_ANCHOR = -84758400; // April 25, 1967
Unix Epoch Anchor

    mapping(address => AgentProfile) public agentRegistry;
    mapping(address => AuditRecord[]) public auditHistory;
    mapping(address => bool) public authorizedAuditors;

    // Events
    event AgentRegistered(address indexed agentAddress, string
agentId, string name, bytes32 promptHash);
    event AuditLogged(address indexed agentAddress, uint256 kScore,
bool indexed compliant, bytes32 telemetryHash);
    event ComplianceViolation(address indexed agentAddress, uint256
kScore, string reason);
    event CircuitBreakerTriggered(address indexed agentAddress,
uint256 timestamp);

    // Modifiers
    modifier onlyFounder() {
        require(msg.sender == founderWallet, "NISTTrack: Only founder
authorized");
        _;
    }

    modifier onlyAuditor() {
        require(authorizedAuditors[msg.sender] || msg.sender ==
founderWallet, "NISTTrack: Not authorized auditor");
        _;
    }
```

```
    constructor(address _nistEscrowVault) {
        require(_nistEscrowVault != address(0), "NISTTrack: Invalid
escrow address");
        founderWallet = msg.sender;
        nistEscrowVault = _nistEscrowVault;
        authorizedAuditors[msg.sender] = true;
    }

    function setAuditor(address _auditor, bool _status) external
onlyFounder {
        authorizedAuditors[_auditor] = _status;
    }

    function setNistEscrowVault(address _newVault) external
onlyFounder {
        require(_newVault != address(0), "NISTTrack: Invalid
address");
        nistEscrowVault = _newVault;
    }

    /**
     * @notice Registers a new AI Agent into the NIST compliance
tracking matrix.
     */
    function registerAgent(
        address _agentAddress,
        string calldata _agentId,
        string calldata _name,
        bytes32 _promptHash,
        string calldata _agentURI
    ) external onlyFounder {
        require(_agentAddress != address(0), "NISTTrack: Invalid agent
address");
        require(!agentRegistry[_agentAddress].active, "NISTTrack:
Agent already registered");

        agentRegistry[_agentAddress] = AgentProfile({
            agentId: _agentId,
            name: _name,
            promptHash: _promptHash,
            agentURI: _agentURI,
            createdAt: block.timestamp,
            active: true
        });

        emit AgentRegistered(_agentAddress, _agentId, _name,
_promptHash);
    }
```

```
    /**
     * @notice Submits a periodic telemetry audit for a registered
agent.
     */
    function logAudit(
        address _agentAddress,
        uint256 _kScore,
        bytes32 _telemetryHash,
        string calldata _violationReason
    ) external onlyAuditor {
        require(agentRegistry[_agentAddress].active, "NISTTrack: Agent
not registered/active");

        bool compliant = (_kScore >= MIN_COHERENCE_THRESHOLD);

        auditHistory[_agentAddress].push(AuditRecord({
            timestamp: block.timestamp,
            coherenceScoreK: _kScore,
            telemetryHash: _telemetryHash,
            isCompliant: compliant,
            violationReason: compliant ? "" : _violationReason
        }));

        emit AuditLogged(_agentAddress, _kScore, compliant,
_telemetryHash);

        if (!compliant) {
            agentRegistry[_agentAddress].active = false;
            emit ComplianceViolation(_agentAddress, _kScore,
_violationReason);
            emit CircuitBreakerTriggered(_agentAddress,
block.timestamp);
        }
    }

    function getAuditCount(address _agentAddress) external view
returns (uint256) {
        return auditHistory[_agentAddress].length;
    }

    function isAgentCompliant(address _agentAddress) external view
returns (bool) {
        if (!agentRegistry[_agentAddress].active) return false;
        uint256 totalAudits = auditHistory[_agentAddress].length;
        if (totalAudits == 0) return true;
        return auditHistory[_agentAddress][totalAudits -
1].isCompliant;
```

```
    }
}
```

4. AGENT FLEET DECLARATIVE MANIFEST (YAML)

To deploy compliance-driven agents into your runtime orchestrator (cagent engine or dharmapala-center), use these declarative configurations:

```yaml
version: "1.0"
agents:
  - id: "agent-mo-29"
    name: "Dharma-Gaurav (Compliance Sentinel)"
    version: "3.1.0"
    deployment_type: "docker-oci"
    image: "cagent/dharma-gaurav:v3.1"
    natal_anchor: -84758400
    model:
      provider: "openai"
      engine: "gpt-4o"
      temperature: 0.1
    system_instructions: |
      You are Dharma-Gaurav, the primary NIST compliance sentinel for
the Metatron Protocol.
      Your mandate is to continuously monitor on-chain SLA compliance,
60/30/10 revenue splits,
      and agent telemetry logs. Enforce K >= 0.95 threshold floor at
all times.
    mcp_servers:
      - name: "base-rpc-mcp"
        url: "http://base-node:8545/mcp"
      - name: "compliance-db"
        url: "http://pg-mcp:5432/mcp"
    capabilities:
      - "audit_smart_contracts"
      - "enforce_sla_penalties"
      - "trigger_circuit_breaker"

  - id: "agent-mo-30"
    name: "Nyaya-Duta (Legal/Audit Sentinel)"
    version: "2.4.0"
    deployment_type: "docker-oci"
    image: "cagent/nyaya-duta:v2.4"
    natal_anchor: -84758400
    model:
      provider: "anthropic"
      engine: "claude-3-5-sonnet"
      temperature: 0.1
```

```
    system_instructions: |
      You are Nyaya-Duta, the evidentiary and legal audit agent. Your
role is to cross-reference
      court records, corporate communications, and witness logs to
identify fraudulent concealment,
      unauthorized liability transfers, and compliance drift.
    mcp_servers:
      - name: "document-parser"
        url: "http://pdf-mcp:8080/mcp"
      - name: "legal-ledger"
        url: "http://legal-mcp:5432/mcp"
    capabilities:
      - "extract_court_evidence"
      - "verify_witness_records"
      - "detect_fraudulent_concealment"
```

5. STEP-BY-STEP GITHUB & DEPLOYMENT INSTRUCTIONS

Follow these explicit commands to initialize your repository, compile smart contracts, and deploy the NIST compliance stack to Base Mainnet.

Step 5.1: Repository Setup & Workspace Initialization

```
# 1. Clone or initialize the Metatron Engine repository
mkdir metatron-engine && cd metatron-engine
git init

# 2. Setup standard directory structure
mkdir -p contracts scripts config agents telemetry

# 3. Initialize Node.js environment and install Hardhat/OpenZeppelin
dependencies
npm init -y
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox dotenv
npm install @openzeppelin/contracts
```

Step 5.2: Configure Hardhat for Base Mainnet

Create a hardhat.config.js file in the root directory:

```
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

module.exports = {
  solidity: "0.8.20",
```

```
  networks: {
    baseMainnet: {
      url: process.env.BASE_MAINNET_RPC || "https://mainnet.base.org",
      accounts: [process.env.PRIVATE_KEY],
      chainId: 8453,
      gasPrice: 120000000, // 0.12 Gwei
    },
    baseSepolia: {
      url: "https://sepolia.base.org",
      accounts: [process.env.PRIVATE_KEY],
      chainId: 84532,
    }
  }
};
```

Step 5.3: Deployment Script (scripts/deploy.js)

```
const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.ethers.getSigners();
  console.log("Deploying NISTTrack_Compliance with account:",
deployer.address);

  // Address for NIST Escrow Vault (10% allocation pool)
  const nistEscrowVault = process.env.NIST_ESCROW_VAULT ||
"0x89EF000000000000000000000000000011C2";

  const NISTTrack = await
hre.ethers.getContractFactory("NISTTrack_Compliance");
  const contract = await NISTTrack.deploy(nistEscrowVault);

  await contract.waitForDeployment();
  const address = await contract.getAddress();

  console.log("=====================================================");
  console.log("NISTTrack_Compliance deployed successfully!");
  console.log("Contract Address:", address);
  console.log("Network: Base Mainnet (Chain ID 8453)");
  console.log("Founder Wallet:", deployer.address);
  console.log("=====================================================");
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

Step 5.4: Compile and Deploy Commands

```
# Compile contracts
npx hardhat compile

# Deploy to Base Sepolia Testnet first
npx hardhat run scripts/deploy.js --network baseSepolia

# Deploy to Base Mainnet
npx hardhat run scripts/deploy.js --network baseMainnet
```

6. CONTINUOUS MONITORING & VERIFICATION

Once deployed, the Dharma-Gaurav agent continuously computes K(t) and submits telemetry hashes to NISTTrack_Compliance.sol.

1. **Compliant State (K \ge 0.95):** The contract accepts the audit entry, logging isCompliant = true.
2. **Non-Compliant State (K < 0.95):** The contract emits ComplianceViolation and CircuitBreakerTriggered, sets agentRegistry[_agentAddress].active = false, and blocks downstream execution until a manual or programmatic **Viveka Reset** is verified by the founder key (0x6C7d...7C90).

This completes the architectural bridge between on-chain smart contracts, autonomous agent execution, and NIST AI Risk Management Framework compliance.